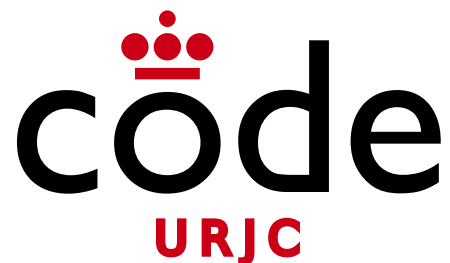


Fundamentos de la Web

Bloque III: Tecnologías de servidor web

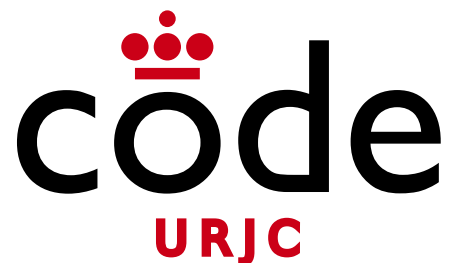
Tema 3.4: Aplicaciones Web con Node



Fundamentos de la Web

Bloque III: Tecnologías de servidor web

Tema 3.4.2: Aplicaciones Web con Imágenes



©2024

Micael Gallego, Francisco Gortázar, Michel Maes, Óscar Soto, Iván Chicano

Algunos derechos reservados

Este documento se distribuye bajo la licencia
“Atribución-CompartirIgual 4.0 Internacional”
de Creative Commons Disponible en
<https://creativecommons.org/licenses/by-sa/4.0/deed.es>

Subida de imágenes

- En las aplicaciones web el usuario puede **subir imágenes** (y cualquier otro tipo de fichero)
- Esas imágenes se **guardan en el disco** o en una **base de datos**
- En disco NO se pueden guardar en la **carpeta public**:
 - No se deben mezclar ficheros estáticos y dinámicos
 - La carpeta **public** puede que sea de sólo lectura cuando la aplicación se ejecuta en **producción** (en un servidor)

Subida de imágenes

- La librería **Multer** gestiona el guardado de subidas de ficheros en la aplicación (*uploads*) se usa la librería *muter*

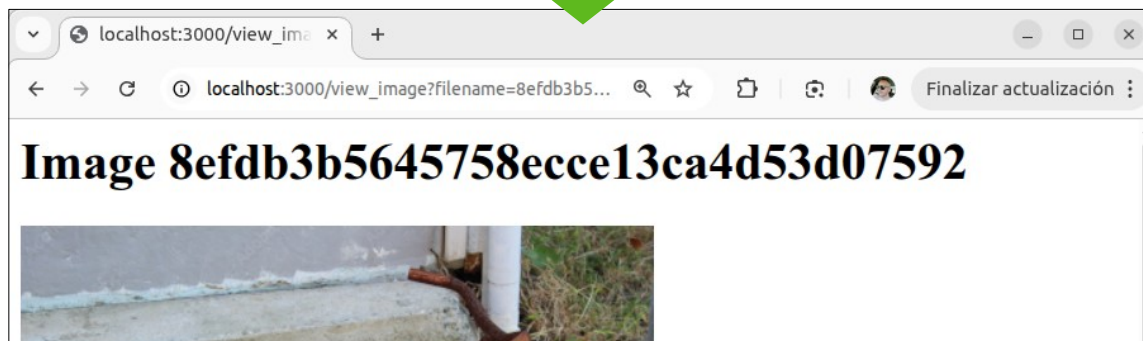
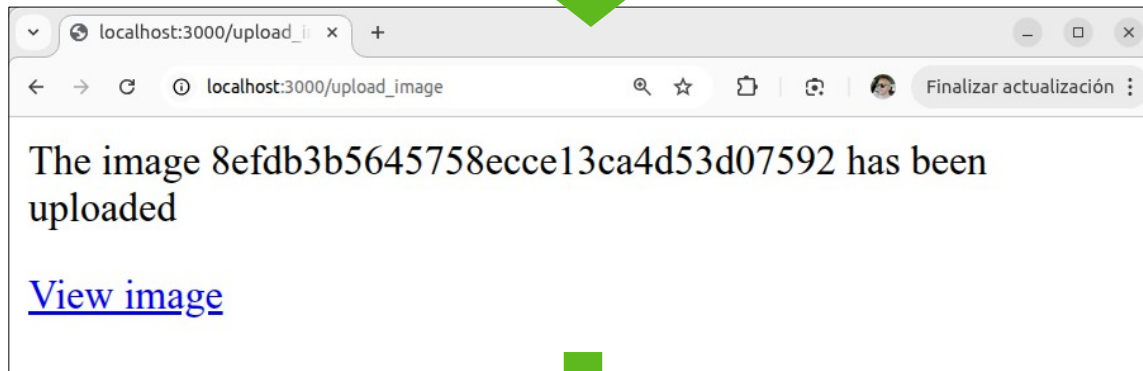
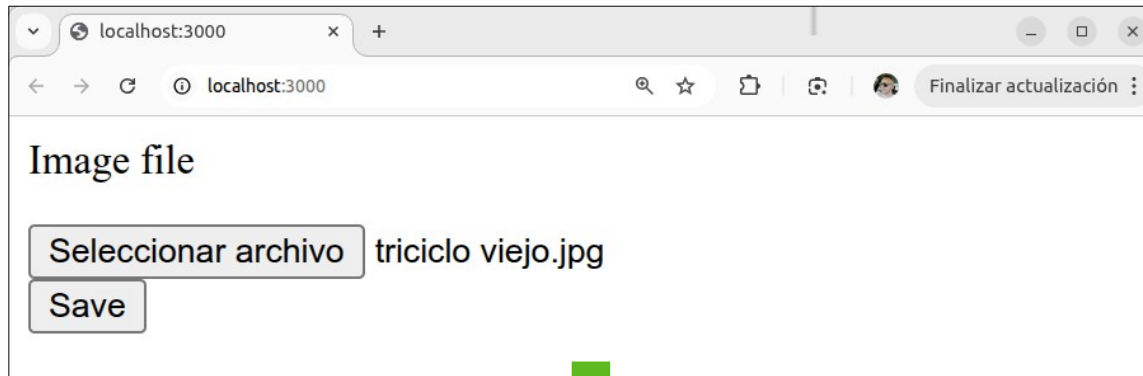
```
$ npm install --save multer
```



```
{
  "name": "web-ejem7",
  "version": "1.0.0",
  "type": "module",
  "dependencies": {
    "express": ...,
    "mustache-express": ...
    "multer": "2.0.2",
  }
}
```

Subida de imágenes

ejem7-images



Subida de imágenes

- Formulario para subir imágenes

public/index.html

```
<html>
  <body>
    <form action="/upload_image" method="post" enctype="multipart/form-data">
      <p>Image file</p>
      <input type='file' name='image' accept=".jpg, .jpeg"/>
      <br>
      <input type='submit' value='Save' />
    </form>
  </body>
</html>
```

Necesario para
subir ficheros

Campo de tipo
fichero

Subida de imágenes

- Función para guardar imágenes

```
import multer from 'multer';  
...  
  
const upload = multer({ dest: 'uploads/' })  
  
router.post('/upload_image', upload.single('image'), (req, res) => {  
  
  if (req.file) {  
    res.render('uploaded_image', { filename: req.file.filename });  
  } else {  
    res.send('File not found!');  
  }  
  
});
```


Subida de imágenes

- Función para guardar imágenes

```
import multer from 'multer';
...

const upload = multer({ dest: 'uploads/' })

router.post('/upload_image', upload.single('image'), (req, res) => {

  if (req.file) {
    res.render('uploaded_image', { filename: req.file.filename });
  } else {
    res.send('File not found!');
  }
})
```

Carpeta para
imágenes subidas

Nombre del campo
file del formulario

Se ha seleccionado
fichero en el form?

Nombre del fichero en la
carpeta /uploads

Subida de imágenes

ejem7-images

- Página de imagen subida

views/uploaded_image.html

```
<html>
<body>

    <p>The image {{filename}} has been uploaded</p>

    <a href="/view_image?filename={{filename}}">View image</a>

</body>
</html>
```

Subida de imágenes

ejem7-images

- Mostrar la imagen guardada

```
router.get('/view_image', (req, res) => {
  res.render('view_image', { filename: req.query.filename });
});
```

views/view_image.html

```
<html>
  <body>
    <h1>Image {{filename}}</h1>
    </img>
  </body>
</html>
```

- **Mostrar la imagen guardada**
 - Como la imagen no está en la carpeta public es necesario implementar una función en el router para descargar la imagen

```
router.get('/image', (req, res) => {  
  let filename = req.query.filename;  
  
  // If filename has \ or / then path traversal attack is detected  
  if(/[\ \/]/.test(filename)){  
    return res.status(400);  
  }  
  
  res.download('uploads/' + filename);  
});
```

Ejercicio 5 – Tablón con imágenes

- Permite asociar una imagen a cada anuncio del Tablón de anuncios (Ejercicio 4)
- Almacena el nombre del fichero como una propiedad del mensaje
- Los mensajes de ejemplo también deben tener imagen
 - Se guardan imágenes en la carpeta /data
 - Se copian a la carpeta uploads al iniciar la app

Ejercicio 5 – Tablón con imágenes

- Guardar imagen

new_post.html

```
<form action="post/new" method="post" enctype="multipart/form-data">
  <p>User: </p><input type='text' name='user' />
  <p>Title:</p><input type='text' name='title' />
  <p>Text:</p><textarea name='text' rows=5 cols=40></textarea>
  <p>Image:</p><input type='file' name='image'>
  <br>
  <input type='submit' value='Save' />
</form>
```

Ejercicio 5 – Tablón con imágenes

- Guardar imagen

src/router.js

```
router.post('/post/new', upload.single('image'), (req, res) => {

  let post = {
    user: req.body.user,
    title: req.body.title,
    text: req.body.text,
    imageFilename: req.file?.filename
  };

  board.addPost(post);

  res.render('saved_post', { id: post.id });

});
```

Ejercicio 5 – Tablón con imágenes

- Descargar imagen

show_post.html

```
...
<p>User: {{post.user}}</p>
<p>Title: {{post.title}}</p>
<p>Text: {{post.text}}</p>

{{#post.imageFilename}}

{{/post.imageFilename}}

<br>
...
```

router.js

```
router.get('/post/:id/image', (req, res) => {
  let post = board.getPost(req.params.id);
  res.download(UPLOADS_FOLDER + '/' + post.imageFilename);
});
```


Ejercicio 5 – Tablón con imágenes

- Eliminar imagen

board.js

```
router.get('/post/:id/delete', (req, res) => {
  let post = board.deletePost(req.params.id);

  if (post && post.imageFilename) {
    //Delete image
    await fs.rm(UPLOADS_FOLDER + '/' + post.imageFilename);
  }

  res.render('deleted_post');
});
```

Borrar la imagen
cuando se borra el post



<https://nodejs.org/api/fs.html>

Ejercicio 5 – Tablón con imágenes

- Imágenes de los datos de demo

load_data.js

```
import fs from 'node:fs/promises';
import * as board from './board.js';

const UPLOADS_FOLDER = './uploads';
const DATA_FOLDER = './data';

let dataFile = 'data.json';

const dataString = await fs.readFile(DATA_FOLDER + '/' + dataFile, 'utf8');

const posts = JSON.parse(dataString);

for(let post of posts){
  board.addPost(post);
}

await fs.rm(UPLOADS_FOLDER, { recursive: true, force: true });
await fs.mkdir(UPLOADS_FOLDER);
await fs.cp(DATA_FOLDER + '/images', UPLOADS_FOLDER, { recursive: true });

console.log('Demo data loaded');
```

Copiar las imágenes a la carpeta upload